目录

0x00. 矩阵.....2
0x01. 矩阵.....2
0x02. 矩阵的加法与数量乘法.....2
0x03. 矩阵乘法.....2
0x04. 矩阵加速递推.....3
0x5. 矩阵的简单应用.....7
0x05.1 zoj 2853 Evolution.....8
0x06. 矩阵表达修改.....9
0x07. 矩阵在图的邻接矩阵中的应用.....9
0x07.1 定长路径统计.....9
0x07.2 定长最短路.....9
0x07.3 限长路径计数/最短路.....9
0x08. Porblem A.....9
0x09. Problem B.....9
0xA. Problem C.....10
0xB. Problem D.....10
0xC. Problem E.....11
0xD. Problem F.....12
0xE. Problem G.....12
0x10. 行列式.....12
0x11. 行列式.....12
0x12. 上三角行列式.....13
0x13. Problem H.....15
0x14. 积和式.....15
0x15. 余子式.....16
0x16. 行列式按一行展开.....16
0x17. 矩阵的逆.....17
0x18. 矩阵的相似.....18
0x19. 矩阵的特征值与特征向量.....18
0x1A. Problem I.....19
0x20. 高斯消元法.....19
0x21. 高斯消元解线性方程组.....20
0x22 高斯-约旦消元法.....22
0x23. 高斯消元解异或线性方程组.....24
0x24. 高斯消元解模线性方程组.....26
0x25. 矩阵求逆.....27
0x26. 行列式计算.....27
0x27. 生成树计数.....28
0x30. 线性空间.....30
0x31. 线性空间.....30
0x32. 线性相关.....30
0x33. 极大线性无关组.....31
0x34. 向量组的秩.....31
0x35. 线性基.....31
0x35.1 前置知识.....31
0x35.2 求线性基.....32
0x35.3 线段树套线性基.....35
0x35.6 线性基全家桶（code）.....37
0x35.7 线性基总结.....41
0x40. 最后的话.....41

0x00. 矩阵

0x01. 矩阵

- 由$n\times m$ 个数排成 n 行，m 列的一张表称为一个 $n\times m$ **矩阵**。
- 矩阵中的每一个数称为这个矩阵的一个元素。
- 矩阵的第i行与第j列交叉位置的元素称为矩阵的(i,j)元。
- 若一个矩阵的行与列相等，则称它为**方阵**。

$$\begin{cases}3x_0+2x_1-5x_2=-1\\2x_0-4x_1+x_2=1\\-x_0-5x_1+x_2=-6\end{cases}\rightarrow\begin{bmatrix}3&2&-5&-1\\2&-4&1&1\\-1&-5&1&-6\end{bmatrix}$$

0x02. 矩阵的加法与数量乘法

设 A 和 B 均为 $n\times m$ 矩阵，且$\forall i\forall jC_{i,j}=A_{i,j}+B_{i,j}$
则称 C 为 A 与 B 的和，记作 $C=A+B$

设 A 为 $n\times m$ 矩阵，且$\forall i\forall jC_{i,j}=k\times A_{i,j}$
则称 C 为 k 与 A 的数量乘积，记作$C=kA$

0x03. 矩阵乘法

设A为$s\times n$矩阵，B为$n\times m$矩阵，令C为$s\times m$矩阵，且

$C_{i,j}=\sum_{k=1}^{n}a_{i,k}\times b_{k,j}$

则称C为A与B的乘积，记作$C=AB$

- 只有左矩阵的列数与右矩阵的行数相同时才能相乘
- 乘积矩阵的 (i,j) 元等于左矩阵的第 i行与右矩阵的第 j 列对应元素乘积之和
- 乘积矩阵的行数等于左矩阵的行数，列数等于右矩阵的列数

为什么我们要这样定义矩阵乘法?
在二维平面上考虑一个点$A(x,y)$
将线段OA逆时针旋转θ，则A点所对应的A'点的坐标是什么?

令 $r=\sqrt{x^2+y^2}$，α为$\angle AOx$，则有

$x=rcos\alpha,\ y=rsin\alpha$
$x'=rcos(\alpha+\theta),y'=rsin(\alpha+\theta)$
$x'=rcos\alpha cos\theta-rsin\alpha sin\theta=xcos\theta-ysin\theta$
$y'=rsin\alpha cos\theta+rcos\alpha sin\theta=xsin\theta+ycos\theta$

$x'=xcos\theta-ysin\theta$
$y'=xsin\theta+ycos\theta$
将方程的系数拿出来排成一张表:

$$A=\begin{bmatrix}\cos\theta&-\sin\theta\\\sin\theta&\cos\theta\end{bmatrix}$$

则该矩阵表示了转角为 θ 的旋转

$$B=\begin{bmatrix}\cos\varphi&-\sin\varphi\\\sin\varphi&\cos\varphi\end{bmatrix}$$

则该矩阵表示了转角为 φ 的旋转

- 考虑转动角度θ+ϕ

$$C=\begin{bmatrix}\cos(\theta+\varphi) & -\sin(\theta+\varphi)\\ \sin(\theta+\varphi) & \cos(\theta+\varphi)\end{bmatrix}$$

$$A=\begin{bmatrix}\cos\theta & -\sin\theta\\ \sin\theta & \cos\theta\end{bmatrix}\qquad B=\begin{bmatrix}\cos\varphi & -\sin\varphi\\ \sin\varphi & \cos\varphi\end{bmatrix}$$

$$c_{1,1}=a_{1,1}\times b_{1,1}+a_{1,2}\times b_{2,1}$$

$$c_{1,2}=a_{1,1}\times b_{1,2}+a_{1,2}\times b_{2,2}$$

$$c_{2,1}=a_{2,1}\times b_{1,1}+a_{2,2}\times b_{2,1}$$

$$c_{2,2}=a_{2,1}\times b_{1,2}+a_{2,2}\times b_{2,2}$$

- 矩阵的乘法不适用交换律
  - 交换后的矩阵甚至不能相乘！即使能够相乘也不一定满足
- 矩阵的乘法适用结合律
  - 直接应用定义展开即可得到
- 矩阵的乘法适用左分配律与右分配律
  - A(B+C)=AB+AC
  - (B+C)D=BD+CD

- 矩阵乘法的结合律是许多题目解题的关键！

一般使用一个二维数组来模拟矩阵。

需要注意的是，因为数组下标是从0开始的，所以我们推出来的矩阵里的下标在实际使用中都要−1。详见下题。

0x04. 矩阵加速递推

AcWing 1303. 斐波那契前 n 项和

大家都知道 Fibonacci 数列吧，$f_1=1,f_2=1,f_3=2,f_4=3,\ldots,f_n=f_{n-1}+f_{n-2}$。

现在问题很简单，输入 n 和 m，求 f_n 的前 n 项和 $S_n \bmod m$。

$$
\begin{aligned}
&f_n = f_{n-1} + f_{n-2}\\
&S_n - S_{n-1} = f_n\\
&= f_{n-1} + f_{n-2}\\
&= S_{n-1} - S_{n-3}\\
&\text{所以 } S_n - S_{n-1} = S_{n-1} - S_{n-3}\\
&S_n = 2 \times S_{n-1} - S_{n-3}
\end{aligned}
$$

那么有递推式了，怎么构造矩阵呢？

> 结论：对于数列 $\{a_n\}$，若 $a_i = k_1 a_{i-1} + k_2 a_{i-2} + \cdots + k_i a_0$，那么
>
> $$\begin{bmatrix} k_1 & k_2 & \cdots & k_{i-1} & k_i \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{bmatrix}^n \begin{bmatrix} a_{i-1} \\ a_{i-2} \\ \vdots \\ a_1 \\ a_0 \end{bmatrix} = \begin{bmatrix} a_{n+i-1} \\ a_{n+i-2} \\ \vdots \\ a_{n+1} \\ a_n \end{bmatrix}$$

https://blog.csdn.net/weixin_45697774

由上述结论，可以构造出矩阵 $A=\begin{bmatrix} 2 & 0 & -1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$，$S=\begin{bmatrix} S_2 \\ S_1 \\ S_0 \end{bmatrix}=\begin{bmatrix} f_0+f_1+f_2 \\ f_0+f_1 \\ f_0 \end{bmatrix}=\begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}$

找到矩阵之后，用矩阵快速幂求出 A^n，然后再乘上矩阵 S，即可求出 S_n

https://blog.csdn.net/weixin_45697774

上述结论证明链接

```
const int N = 3;
int n, m;
int a[N][N];
int f[N];
void mul(int c[], int a[], int b[][N])
{
    int tmp[N] = {0};
    for(int j = 0; j < N; ++ j)//2的行
        for(int k = 0; k < N; ++ k)//1的列2的行，1的列=2的行
            //只有一行，求这一行的每一列
            tmp[j] = (tmp[j] + (ll)a[k] * b[k][j]) % m;
    memcpy(c, tmp, sizeof tmp);
}
void mul(int c[][N], int a[][N], int b[][N])
{
    int tmp[N][N] = {0};
    for(int i = 0; i < N; ++ i)
        for(int j = 0; j < N; ++ j)
            for(int k = 0; k < N; ++ k)
```

```
                tmp[i][j] = (tmp[i][j] + (ll)a[i][k] * b[k][j]) % m;
    memcpy(c, tmp, sizeof tmp);
}

int main()
{
    scanf("%d%d", &n, &m);
    int f1[N] = {1, 1, 1};
    int a[N][N] = {
        0, 1, 0,
        1, 1, 1,
        0, 0, 1
    };
    n -- ;
    //这里是从第一项开始的，所以要--;
    //就是说数组模拟，是从f[0]开始的，f[0]就是斐波那契数列里的f_1
    //所以我们输出的是f2而不是式子中的f3
/*  若求的是斐波那契第n项的值，只需要把f和a改成下面的矩阵，n不需要-1，最后输出f[0]即可。
    int f[N] = {0, 1};
    int a[N][N] = {{0, 1}, {1, 1}};
    printf("%d\n", f[0]);
*/
    while(n){
        if(n & 1) mul(f1, f1, a);
        mul(a, a, a);
        n >>= 1;
    }
    printf("%d\n", f1[2]);
    return 0;
}
```

AcWing 1304. 佳佳的斐波那契

佳佳对数学，尤其对数列十分感兴趣。

在研究完 Fibonacci 数列后，他创造出许多稀奇古怪的数列。

例如用 $S(n)$ 表示 Fibonacci 前 n 项和 $\bmod m$ 的值，即 $S(n)=(F_1+F_2+\ldots+F_n) \bmod m$，其中 $F_1=F_2=1, F_i=F_{i-1}+F_{i-2}$。

可这对佳佳来说还是小菜一碟。

终于，她找到了一个自己解决不了的问题。

用 $T(n)=(F_1+2F_2+3F_3+\ldots+nF_n) \bmod m$ 表示 Fibonacci 数列前 n 项变形后的和 $\bmod m$ 的值。

现在佳佳告诉你了一个 n 和 m，请求出 $T(n)$ 的值。

https://blog.csdn.net/weixin_45697774

fib 1 1 2 3 5 8 13 ...

$f_1=1\ f_2=1\ f_n=f_{n-2}+f_{n-1}$

$$F_n=[f_n,f_{n+1}]_{1\times2}\times\begin{bmatrix}0&1\\1&1\end{bmatrix}_{2\times2}=F_{n+1}=[f_{n+1},f_{n+2}]$$

$F_n\cdot A=F_{n+1}$

$F_n\cdot A^2=F_{n+2}$

$F_n=F_1\cdot A^{n-1}$ 其中 $F_1=[f_1,f_2]=[1,1]$

$F_n=F_0\cdot A^n$

$S_{n+1}=S_n+f_{n+1}$ $S_n=f_1+f_2+\cdots f_n$.

$$F_n=[f_n,f_{n+1},S_n]\cdot\begin{bmatrix}0&1&0\\1&1&1\\0&0&1\end{bmatrix}_{3\times3}=F_{n+1}=[f_{n+1},f_{n+2},S_{n+1}]$$

$F_n=F_1\cdot A^{n-1}$ ⇒ A一定全部为常量

不能有变量(每个A必须相同)

$T_n=1\cdot f_1+2\cdot f_2+\cdots+n\cdot f_n$ 因为 T_n 有变量,我们无法直接构造出A使得 $T_n\cdot A=T_{n+1}$

∴考虑曲线求国

$P_n=n\cdot S_n-T_n=(n-1)f_1+(n-2)f_2+\cdots f_{n-1}$ ①

$P_{n+1}=(n+1)\cdot S_{n+1}-T_{n+1}=n\cdot f_1+(n-1)f_2+\cdots f_n$ ②

$P_{n+1}-P_n=②-①=f_1+f_2+\cdots+f_n=S_n\rightarrow P_{n+1}=P_n+S_n\rightarrow P_n=P_{n-1}+S_{n-1}$

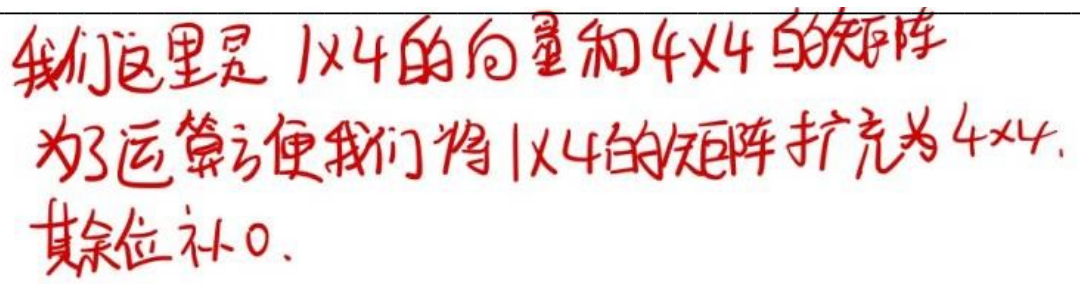
$P_n=n\cdot S_n-T_n\Rightarrow T_n=n\cdot S_n-P_n$

$P_n=P_{n-1}+S_{n-1}$
$S_n=S_{n-1}+f_n$
$f_n=f_{n-1}+f_{n-2}$
} 左右相无变量

∴我们构造出一个新 $F_n=[f_n,f_{n+1},S_n,P_n]\cdot\begin{bmatrix}0&1&0&0\\1&1&1&0\\0&0&1&1\\0&0&0&1\end{bmatrix}_{4\times4}=F_{n+1}=[f_{n+1},f_{n+2},S_{n+1},P_{n+1}]$

$F_{n+1}=F_n\cdot A$

$\Rightarrow F_n=F_1\cdot A^{n-1}$ $F_1=[1,1,1,0]$ $(f_1=1,f_2=1,S_1=S_0+f_1=1,P_1=P_0+S_0=0)$



https://blog.csdn.net/weixin_45697774

计算公式为：$T_n = n * S_n - P_n$

```cpp
const int N = 4;
int n, m;
int f[N][N];
void mul(int c[][N], int a[][N], int b[][N])
{
    static int tmp[N][N];
    memset(tmp, 0, sizeof tmp);
    for(int i = 0; i < N; ++ i)
        for(int j = 0; j < N; ++ j)
            for(int k = 0; k < N; ++ k)
                tmp[i][j] = (tmp[i][j] + (ll)a[i][k] * b[k][j]) % m;
    memcpy(c, tmp, sizeof tmp);
}

int main()
{
    scanf("%d%d", &n, &m);
    // {fn, fn+1, sn, pn}
    // pn = n * sn - tn
    int f1[N][N] = {1, 1, 1, 0};
    int a[N][N] = {
        {0, 1, 0, 0},
        {1, 1, 1, 0},
        {0, 0, 1, 1},
        {0, 0, 0, 1},
    };
    int k = n - 1;
    while(k){
        if(k & 1)mul(f1, f1, a);
        mul(a, a, a);
        k >>= 1;
    }
    cout << (((ll)n * f1[0][2] - f1[0][3] % m + m) % m) << endl;
    return 0;
}
```

0x5. 矩阵的简单应用

- 将行列式的变换动作构造成一个矩阵（如 平移 旋转 翻转 等操作）
- 将一个一次递推式 构造出 矩阵。
  例如：$fibonacci$ 数列的递推关系 $f(n) = f(n-1) + f(n-2)$
  我们可以将矩阵构造为：

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} b \\ a+b \end{pmatrix}$$

0x05.1 zoj 2853 Evolution

题目：

> Evolution is a long, long process with extreme complexity and involves many species. Dr. C. P. Lottery is currently investigating a simplified model of evolution: consider that we have N (2 <= N <= 200) species in the whole process of evolution, indexed from 0 to N -1, and there is exactly one ultimate species indexed as N-1. In addition, Dr. Lottery divides the whole evolution process into M (2 <= M <= 100000) sub-processes. Dr. Lottery also gives an 'evolution rate' P(i, j) for 2 species i and j, where i and j are not the same, which means that in an evolution sub-process, P(i, j) of the population of species i will transform to species j, while the other part remains unchanged.
>
> Given the initial population of all species, write a program for Dr. Lottery to determine the population of the ultimate species after the evolution process. Round your final result to an integer.

https://blog.csdn.net/weixin_45697774

它就是先进行构造“进化”这个动作的矩阵。

$$有多少次进化 = 初始物种状态矩阵 \times (进化矩阵)^n$$

```cpp
#include<cstdio>
#include<cstring>
int n;
struct matrix
{
    double a[200][200];
} ans, base;

matrix multiply(matrix x, matrix y)
{
    matrix tmp;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
        {
            tmp.a[i][j] = 0;
            for (int k = 0; k < n; k++) tmp.a[i][j] += x.a[i][k] * y.a[k][j];
        }
    return tmp;
}
void fast_mod(int k)
{
    while (k)
    {
        if (k & 1) ans = multiply(ans, base);
        base = multiply(base, base);
        k >>= 1;
    }
}
int main()
{
    int m, t, xi, yi;
    double s, zi, fin;
    int num[205];
    while (~scanf("%d%d", &n, &m) && !(n == 0 && m == 0))
    {
        for (int i = 0; i < n; i++) scanf("%d", &num[i]);
        scanf("%d", &t);
```

```
        memset(base.a, 0, sizeof(base.a));
        for (int i = 0; i < n; i++) base.a[i][i] = 1;    //初始为进化成自己
        while (t--)
        {
            scanf("%d%d%lf", &xi, &yi, &zi);
            base.a[xi][yi] += zi;
            base.a[xi][xi] -= zi;
        }
        memset(ans.a, 0, sizeof(ans.a));
        for (int i = 0; i < n; i++) ans.a[i][i] = 1;
        fast_mod(m);
        fin = 0;
        for (int i = 0; i < n; i++)
            fin += num[i] * ans.a[i][n-1];
        printf("%.0f\n", fin);
    }
    return 0;
}
```

0x06. 矩阵表达修改

0x07. 矩阵在图的邻接矩阵中的应用

0x07.1 定长路径统计

0x07.2 定长最短路

0x07.3 限长路径计数/最短路

0x08. Porblem A

幽鬼是游戏Dotb的一个英雄，有个被动技能“折射”，可以将受到的伤害的一定百分比平均反弹给周围的其他英雄

现在有N个幽鬼，每个幽鬼有不同折射率。给出每个幽鬼可以折射到的英雄与幽鬼的折射率，并在一开始对第K个幽鬼造成P点伤害，求最后每个幽鬼受到的伤害。

例：若1号幽鬼能反弹到2号幽鬼，折射率为60%，2号幽鬼不能反弹到1号幽鬼，且在最初给予1号幽鬼400点伤害，则最后1号幽鬼受到160点伤害，2号幽鬼受到240点伤害
折射进行无数次
$N <= 100$

Solution

设第i个幽鬼受到的**总伤害**为X_i
则对于每个幽鬼可以列出一个关于伤害的方程，共N个方程
解方程组即可。
$O(N^3)$

0x09. Problem B

给定$A[0]$，$A[1]$，$A[i] = 3 \times A[i-1] - 2 \times A[i-2] (i > 1)$，求$A[N]$。
答案模1000000007输出，$N <= 10^9$

Solution

$$[a_i \ \ a_{i+1}]\times\begin{bmatrix}0 & -2\\ 1 & 3\end{bmatrix}=[a_{i+1} \ \ -2a_i+3a_{i+1}]=[a_{i+1}\ a_{i+2}]$$

- 即有 $[a_n\ a_{n+1}]=[a_0\ a_1]\times\begin{bmatrix}0 & -2\\ 1 & 3\end{bmatrix}^n$

- 应用快速幂进行计算
- 快速幂可以在 $O(c^3\times\log N)$ 的时间复杂度内计算出一个c阶矩阵的N次幂

形如$F[i]=A[1]*F[i-1]+A[2]*F[i-2]+\ldots+A[c]*F[i-c]$的方程均可以利用矩阵乘法和快速幂进行优化

直接求解的时间复杂度为$O(cN)$

使用矩阵乘法的时间复杂度为$O(c^3logN)$

适用c较小而N较大的情况

代码同上面的矩阵快速幂

oxA. Problem C

在一个古老的国家对数字有这样的规定：这个国家适用$0,1,2,3,4,5,6,7,8,9$这10个数字，但规定有P个整数对（xi,yi）$(0<=xi,yi<=9)$不能连续出现。
一个整数对(xi,yi)表示在一个数中，xi的后面不能紧跟着yi
现在我们要知道在这个规定下，第K小的正整数的前50位是多少。
$K<=2*10^9$

Solution

- 不妨设f[i,j]表示i位数，第1位为j的数的个数
- 则 $f[i,j]=\sum_{k=0..9,\text{k可以在j后}} f[i-1,k]$

- 注意到位数会很大
- 二分答案的位数t，从依次计算f[t]，f[t-1],...,f[t-50]，以确定每一位的值
- 时间复杂度 $O(c^3\log^2 K)$,其中c在这里为10

oxB. Problem D

给一张 N 个点的有向图，每条边的权值是1 ~ 5之间的整数，求从 S 走到 T 长度为 K 的路径条数，答案模1000003输出，$N<=10,K<=10^9$

Solution

有如下重要结论：
考虑一张无权的邻接矩阵 A
设$B=A^2$，则B_{ij} 表示从 i 点到j 点，走过两条边的路径条数

稍加扩展即可得到：

设$C=A^k$，则C_{ij}表示从 i 点到 j 点，走过 k 条边的路径条数

这是路径长度为1的情况

由于路径长度为1 ~ 5之间的整数，可以拆点后再使用上述结论

时间复杂度$O((5N)^3logK)$

oxC. Problem E

bzoj2676 Contra

给定N，R，Q，S
有N个关卡，初始有 Q 条命，且任意时刻最多只能有 Q 条命

每通过一个关卡，会得到 u 分和 1 条命，其中$u=min\{$ 最近一次连续通过的关数 , $R\}$
若没有通过这个关卡，将失去一条命，并进入下一个关卡

若没有生命或N个关卡均已挑战过一次时，游戏结束，得到的分数为每关得到的分数的总和
每条命通过每个关卡的概率为$p(0<=p<=1)$，原先最高分纪录为S
求当p至少为多少时，期望获得的总分能够超过最高分。

$1<=N<=10^8,1<=R<=20,1<=Q<=5$，输出保留6位小数

Solution

例如，当N=12，R=3，Q=2时
第一关：未通过 u=0 获得分数0 总分为0 剩余生命1
第二关：通过 获得分数1 总分为1 剩余生命2
第三关：通过 获得分数2 总分为3 剩余生命2
第四关：通过 获得分数3 总分为6 剩余生命2
第五关：通过 获得分数3 总分为9 剩余生命2
第六关：未通过 获得分数0 总分为9 剩余生命1
第七关：通过 获得分数1 总分为10 剩余生命2
第八关：未通过 获得分数0 总分为10 剩余生命1
第九关：未通过 获得分数0 总分为10 剩余生命0
游戏结束 总分为10

容易想到二分答案p
令$g[i，j，k]$表示在第 i 关，u 值为 j ，还剩 k 条生命的概率

$g[i+1，min(j+1,R)，min(k+1,Q)]+=g[i，j，k]\times p$

$g[i+1，0，k-1]+=g[i，j，k]\times(1-p)$

$Answer+=g[i，j，k]\times p\times(j+1)$

将后两维展开为一维数组进行矩阵乘法
复杂度为$O(T(RQ)^3logN)$，其中T为二分的次数。

这个复杂度仍然过大，需要继续优化

我们发现，当$u>=Q-1$时，生命个数一定是Q

大量状态无用

去掉无用的状态后，矩阵的边长由PQ级别降为$P+Q$级别

再进行矩阵乘法，时间复杂度为$O(T(R+Q)^3logN)$，其中T为二分次数。

这样可以通过所有的测试数据

☼即使使用时间复杂度较大的算法，加入常数优化也可以通过所有测试数据。

0xD. Problem F

给定N个点的无权有向图，询问长度$< K$的环有几个，答案模M输出，$N <= 35K <= 1000000$

Solution

令$f[k，i，j]$表示长度为k，从i走到j的方案数

则答案为
$\sum_{i=0}^{n-1}\sum_{j=2}^{k-1}F[j,i,i]$

暴力计算？$O(KN^3)$

将矩阵直接展开进行优化？$O(N^6logk)$

将K按二进制位分为若干个子问题
容易算出 2^i 的方案数的和
容易算出一个长度 k 时的方案数
将其相乘，再将子问题的答案求和即可
时间复杂度$O(N^3logk)$

0xE. Problem G

给定$A[0],A[1]，A[i]=3\times A[i-1]-2\times A[i-2](i>1)$，求$A[N]$。
，答案模10^{100}输出,$N <= 10^9$

Solution

- 与problem one的区别在于模数为高精度数
- 直接矩阵乘法将涉及高精度数的相乘
- 实际上，我们可以改写矩阵

$$\begin{bmatrix}0 & -2\\1 & 3\end{bmatrix}=\begin{bmatrix}-2 & -1\\1 & 1\end{bmatrix}\begin{bmatrix}1 & 0\\0 & 2\end{bmatrix}\begin{bmatrix}-1 & -1\\1 & 2\end{bmatrix}$$

$$\begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}=\begin{bmatrix}-2 & -1\\1 & 1\end{bmatrix}\begin{bmatrix}-1 & -1\\1 & 2\end{bmatrix}$$

- 幂变得容易求得

https://blog.csdn.net/weixin_45697774

构造这样的矩阵，可以利用矩阵的特征值来完成

需要用到线性代数的一些简单知识

0x10. 行列式

0x11. 行列式

$$\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix} = \sum_{j_1 j_2 \cdots j_n} (-1)^{r(j_1 j_2 \cdots j_n)} a_{1 j_1} a_{2 j_2} \cdots a_{n j_n},$$

其中$j_1 j_2 \cdots j_n$是n元排列，

$r(j_1 j_2 \cdots j_n)$表示其中的逆序对个数

N=2,3

$$\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = a_{11} \times a_{22} - a_{12} \times a_{21}$$

$$\begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} = a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32}$$

$$- a_{13}a_{22}a_{31} - a_{12}a_{21}a_{33} - a_{11}a_{23}a_{32}$$

0x12. 上三角行列式

- 主对角线下方全为0的n阶行列式称为上三角行列式
- 上三角行列式的值为对角线上N个元素的乘积

$$\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ 0 & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{vmatrix} = a_{11}a_{22} \cdots a_{nn}$$

直接使用定义 $O(n!)$
状态压缩DP $O(2^n)$

我们需要一个更优秀的算法来解决这个问题！

- 性质一
- 交换行列式的行与列，行列式值不变

$$\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix} = \begin{vmatrix} a_{11} & a_{21} & \cdots & a_{n1} \\ a_{12} & a_{22} & \cdots & a_{n2} \\ \vdots & \vdots & & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{nn} \end{vmatrix}$$

- 性质二
- 行列式一行的因子可以提出

$$\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ ka_{21} & ka_{22} & \cdots & ka_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix} = k \begin{vmatrix} a_{11} & a_{21} & \cdots & a_{n1} \\ a_{12} & a_{22} & \cdots & a_{n2} \\ \vdots & \vdots & & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{nn} \end{vmatrix}$$

- 性质三
- 两行互换，行列式反号

$$\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix} = \begin{vmatrix} a_{21} & a_{22} & \cdots & a_{2n} \\ a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{nn} \end{vmatrix}$$

- 性质四
- 将一行的倍数加到另一行上，行列式的值不变

$$\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{vmatrix} = \begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21}+ka_{11} & a_{22}+ka_{12} & \cdots & a_{2n}+ka_{1n} \\ \vdots & \vdots & & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{nm} \end{vmatrix}$$

利用性质三和性质四

我们可以用类似高斯消元的方法

将一个普通的行列式通过初等行变换转化为一个上三角行列式进行求值

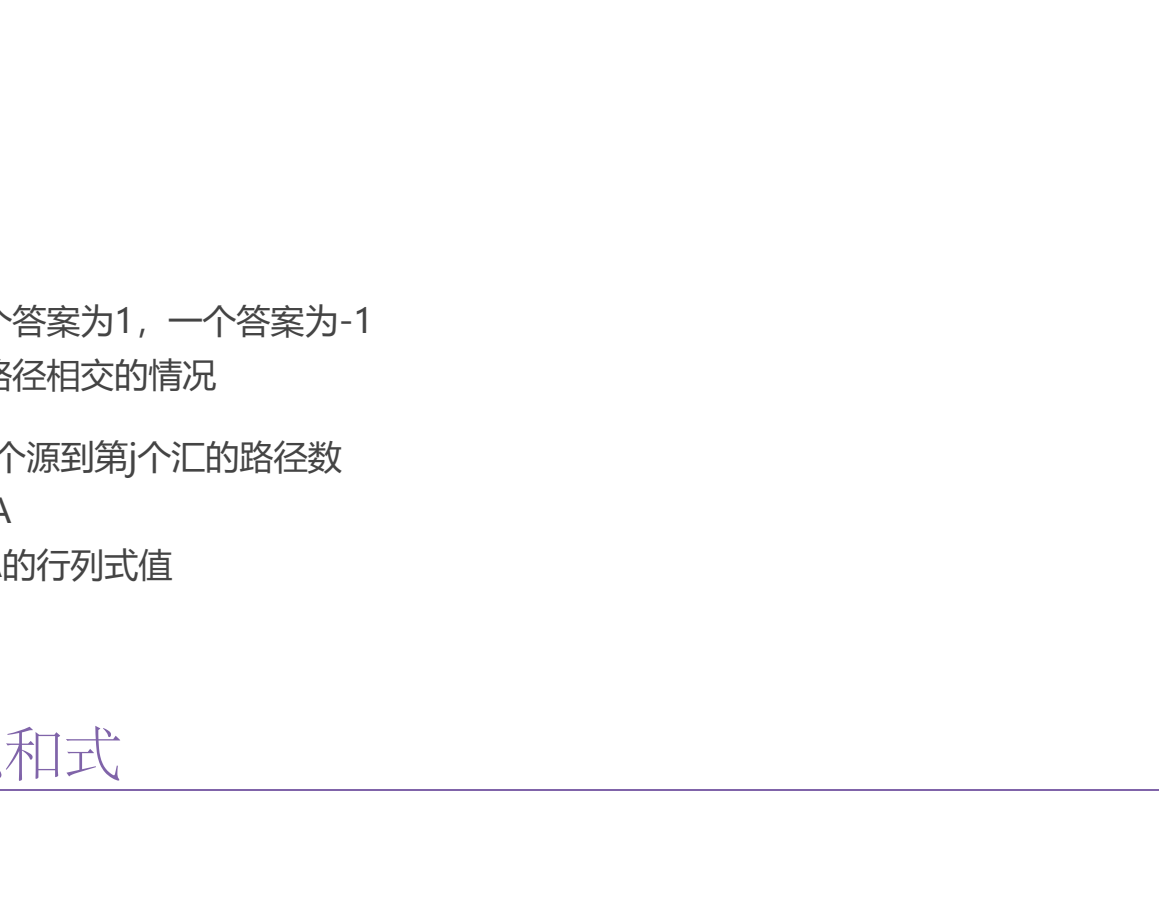
$O(N^3)$

代码见0x26节

0x13. Problem H

给定一张N个点M条边的有向无环图
保证有恰好K个点入度为0，我们称之为源；K个点出度为0，我们称之为汇，将源与汇的K个点分别标号为$1..K$
从图中选出K条没有公共点的路径，每条路径经过一个源与一个汇。不妨设从源i出发的边到达了汇T_i。
对于任意i,j，若有i<j且$T_i > T_j$，我们称之为一个逆序对。
如果所有路径中的逆序对个数是偶数，答案增加1，否则答案减少1
求答案模一个素数p后的值
$N <= 600$，$M <= 10^5$，$2 <= p <= 1000000007$

Solution



注意到相交的路径不影响答案
e.g.
1->2->3
4->2->5
与
1->2->5
4->2->3
成对出现，且一个答案为1，一个答案为-1
因此不需要考虑路径相交的情况

令A[i,j]表示从第i个源到第j个汇的路径数
简单dp即可得到A
易发现答案即为A的行列式值
$O(N^3 + NM)$

0x14. 积和式

$$\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix} = \sum_{j_1 j_2 \cdots j_n} a_{1 j_1} a_{2 j_2} \cdots a_{n j_n},$$

其中$j_1 j_2 \cdots j_n$是n元排列

- 行列式的无系数版本
- 不容易计算
- 对同一个方阵，积和式的奇偶性与行列式相同

0x15. 余子式

- 将行列式A划去第i行与第j列，得到的N-1阶行列式称为余子式，通常记作Mij

- 令 $A_{ij} = (-1)^{i+j} M_{ij}$，我们称Aij为A的(i,j)元的代数余子式

0x16. 行列式按一行展开

$$|A|=\sum_{j=1}^{n} a_{ij}A_{ij}$$

- 思考：fibonacci数列的第N项如何用一个元素仅含0,1,-1的N阶行列式表示?

Solution

Theorem 2 The general term of Fibonacci sequence of numbers can be expressed by the following sequence of determinants of order n:

$$F_{n+1}=\begin{vmatrix} 1 & 1 & 0 & \cdots & 0 & 0 \\ -1 & 1 & 1 & \cdots & 0 & 0 \\ 0 & -1 & 1 & \cdots & 0 & 0 \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ 0 & 0 & 0 & \cdots & 1 & 1 \\ 0 & 0 & 0 & \cdots & -1 & 1 \end{vmatrix}_n$$

$$n=1,2,3,\cdots$$

Proof Observe first that for $n\geqslant 1$ and $a,b,c\in R$, we have

$$F_{n+1}(b,a,c)=\begin{vmatrix} a & c & 0 & \cdots & 0 & 0 \\ b & a & c & \cdots & 0 & 0 \\ 0 & b & a & \cdots & 0 & 0 \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ 0 & 0 & 0 & \cdots & a & c \\ 0 & 0 & 0 & \cdots & b & a \end{vmatrix}_n =$$

$$\begin{cases} (1+n)\left(\frac{a}{2}\right)^n & \text{if } a^2=4bc \\ \dfrac{\left(a+\sqrt{a^2-4bc}\right)^{n+1}-\left(a-\sqrt{a^2-4bc}\right)^{n+1}}{2^{n+1}\sqrt{a^2-4bc}} & \text{if } a^2\neq 4bc \end{cases}$$

Therefore

$$F_{n+1}=F_{n+1}(-1,1,1)=$$

$$\frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^{n+1}-\left(\frac{1-\sqrt{5}}{2}\right)^{n+1}\right] \quad n=1,2,3,\cdots$$

which is the general term of Fibonacci sequence of numbers.

来自论文：斐波那契数列的矩阵和行列式表示

0x17. 矩阵的逆

若$A\times B=E$，则我们称B是A的逆，记作$B=A^{-1}$

行列式值不为0的n阶矩阵必有逆矩阵
如何求逆矩阵?

将A通过初等行变换变为单位矩阵，再将单位矩阵按同样顺序做相同的初等行变换，则得到的矩阵即为A^{-1}。

我们可以利用高斯消元解决矩阵求逆的问题。

```
struct M {
    ll f[NN][NN * 2];
    int Gauss() {
        for (int i = 1; i ≤ n; i++)
            f[i][i + n] = 1;
        for (int i = 1; i ≤ n; i++) {
            for (int j = i; j ≤ n; j++)
                if (f[j][i]) {
                    for (int k = 1; k ≤ nn; k++)
                        swap(f[i][k], f[j][k]);
                    break;
                }
            if (!f[i][i]) {
                return 0;
            }
            ll inv, y;
            exgcd(f[i][i], mod, inv, y);
            inv = (inv % mod + mod) % mod;
            for (int j = i; j ≤ nn; j++)
                f[i][j] = f[i][j] * inv % mod;
            for (int j = 1; j ≤ n; j++)
                if (j ≠ i) {
                    y = f[j][i];
                    for (int k = i; k ≤ nn; k++)
                        f[j][k] = (f[j][k] - y * f[i][k] % mod + mod) % mod;
                }
        }
        return 1;
    }
} m;
```

0x18. 矩阵的相似

- 设A，B为n级矩阵
- 若存在矩阵P，使$A = P^{-1}BP$，则称A与B相似

- 为什么要研究相似?
- 可以优化一部分矩阵的计算
- $A^m = (P^{-1}BP)^m = P^{-1}B^mP$

0x19. 矩阵的特征值与特征向量

- 设A是n级矩阵
- 若有非零列向量α，使Aα=λα，且λ∈K
- 则称λ是A的一个特征值，α是A的属于特征值λ的一个特征向量

- λ为A的特征值的充要条件：$|\lambda I - A| = 0$

0x1A. Problem I

一个培养皿分为N个区域
一开始第1个区域有一个细胞
在每个单位时间，第i个区域的细胞，每个细胞会分裂成为K个细胞，且进入第j个区域的细胞有A_{ij}个。
细胞只会进入区域编号大于等于当前区域编号的区域，即
$\forall i, \forall j < i, A_{i,j} = 0$
数据保证$\forall i \forall j\ A_{i,j} \neq A_{i,j}$
求T个时刻后细胞的数目，答案模p输出
$N <= 100$

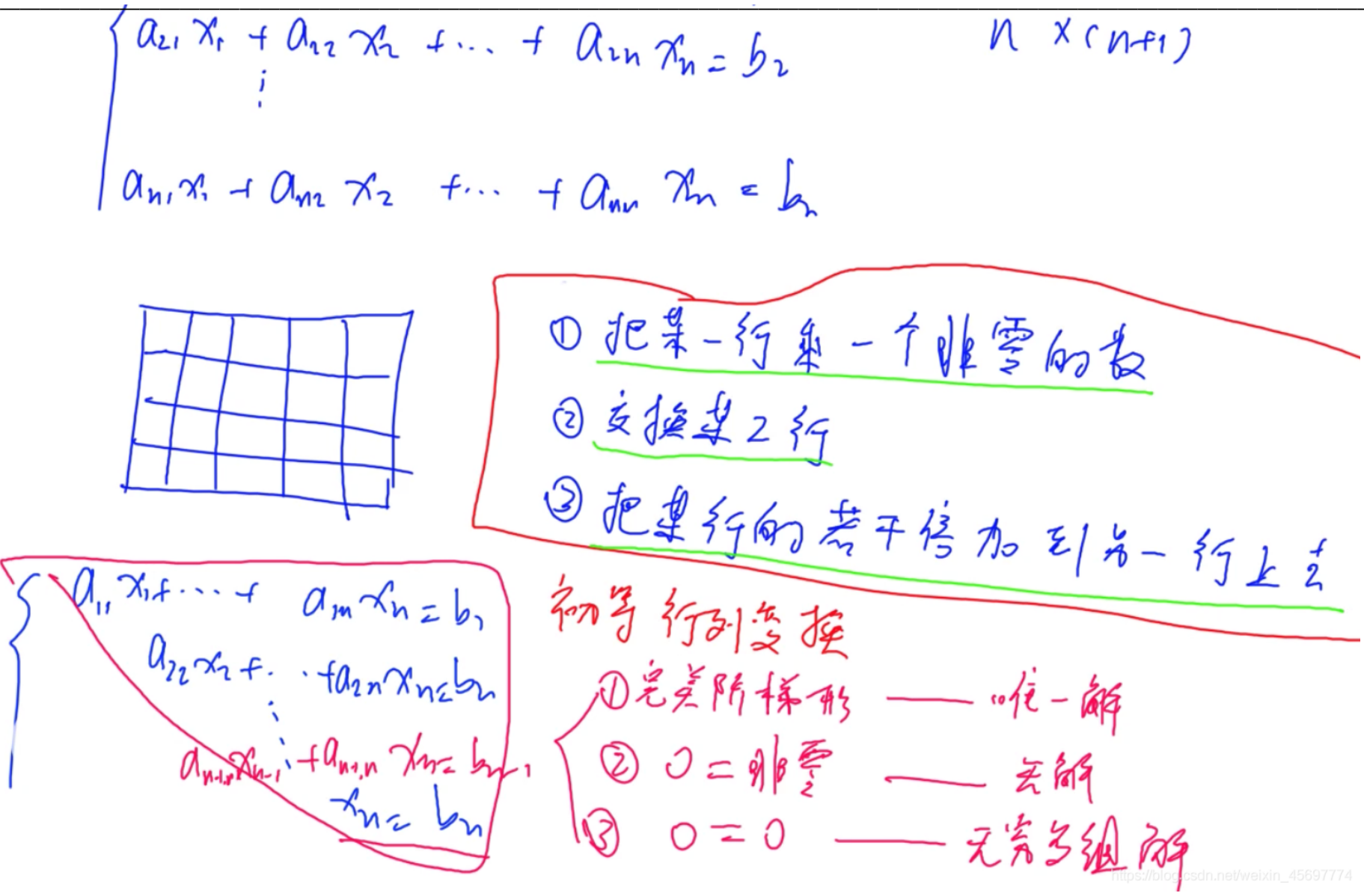
Solution

直接矩阵乘法 $O(N^3 logT)$
注意到是上三角矩阵，且对角元两两不同
特征值即为对角线上的N个元素
我们可以在$O(N^3)$时间内求出所有特征值对应的特征向量
利用矩阵的相似将A写成三个矩阵相乘的形式，只需要对对角的元素使用快速幂
时间复杂度$O(N^3)$
需要高精度时时间复杂度大大降低

0x20. 高斯消元法

高斯消元用于解线性方程组
线性方程组如下：

$$\left.\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots & \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m, \end{aligned}\right\}$$



> 定理：增广矩阵$R(\bar{A})$经过初等行变换不改变最后线性方程组的解（增广矩阵就是系数矩阵并上常数矩阵）

高斯消元算法总结起来就是一句话：通过初等行变换把增广矩阵的线性方程组变为简化阶梯型矩阵的线性方程组求解算法。

具体的思想就是：对于每一个未知量x_i，找到一个x_i 的系数非零，但是x_1 ~ x_{i-1}的系数都是零的方程，然后用初等行变换把其他方程的x_i的系数全部消成零。

齐次线性方程一定有零解

$n = m$时方程组

仅有零解 $\Leftrightarrow D = |A| \neq 0 \Leftrightarrow R(A) = n$

有非零解 $\Leftrightarrow D = |A| = 0 \Leftrightarrow R(A) < n$

非齐次方程

有唯一解 $\Leftrightarrow R(A) = R(\bar{A}) = r = n$

有无穷多解 $\Leftrightarrow R(A) = R(\bar{A}) = r < n$

无解 $\Leftrightarrow R(A) \neq R(\bar{A})$

高斯消元 $\Leftrightarrow$ 消成上三角矩阵

高斯-约旦消元 $\Leftrightarrow$ 消成对角矩阵

0x21. 高斯消元解线性方程组

输入一个包含n个方程n个未知数的线性方程组。

方程组中的系数为实数。

求解这个方程组。

下图为一个包含m个方程n个未知数的线性方程组示例：

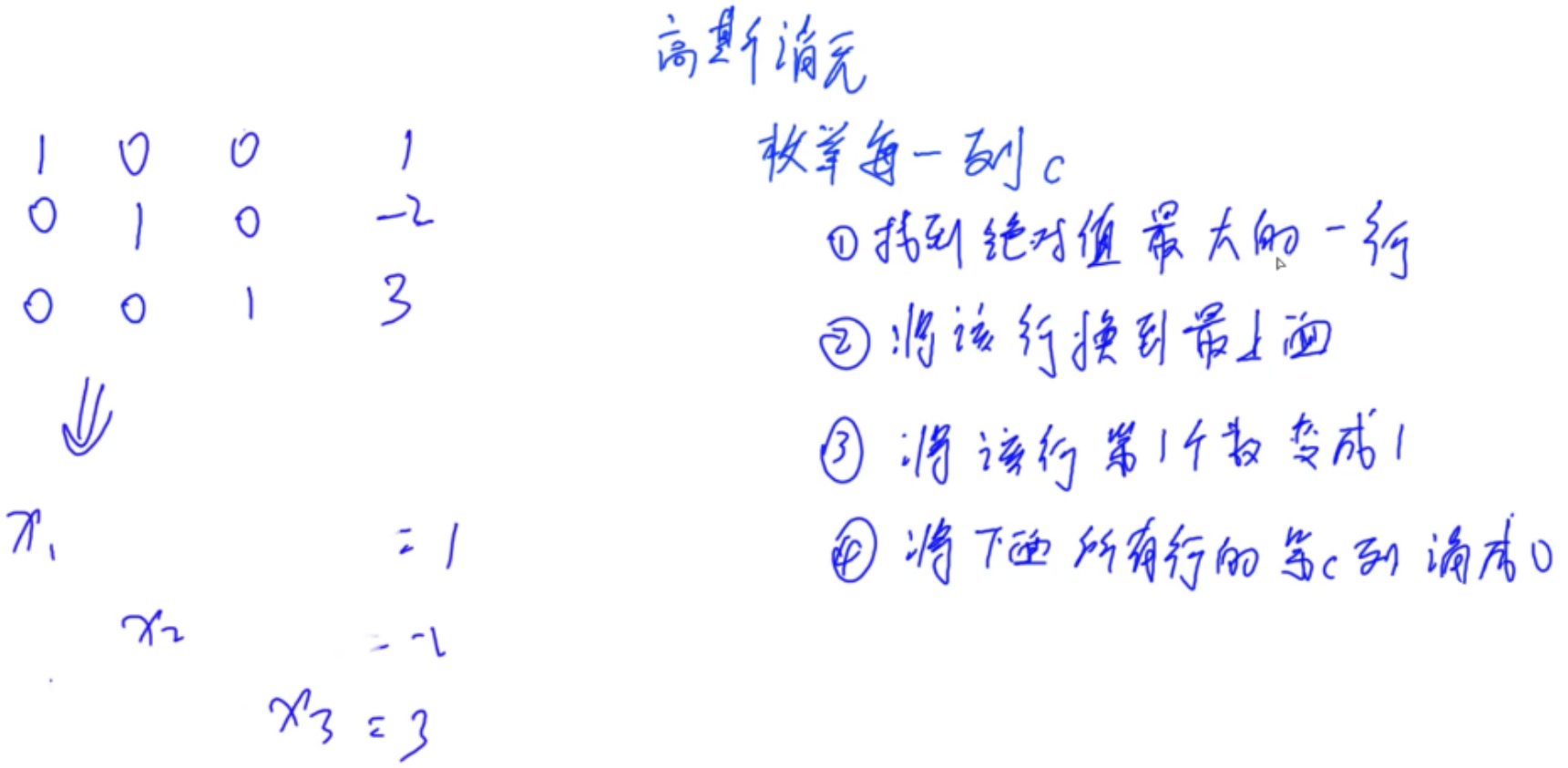
$$\left.\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots & \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m, \end{aligned}\right\}$$

https://blog.csdn.net/weixin_45697774

如果给定线性方程组存在唯一解，则输出共n行，其中第i行输出第i个未知数的解，结果保留两位小数。

如果给定线性方程组存在无数解，则输出“Infinite group solutions”。

如果给定线性方程组无解，则输出“No solution”。



https://blog.csdn.net/weixin_45697774

步骤：

- 增广矩阵行初等行变换为行最简形；
- 还原线性方程组；
- 求解第一个变量；
- 补充自由未知量；
- 列表示方程组通解。

```
const int N = 507, mod = 1e9 + 7;
const double eps = 1e-6;
int n, m;
double a[N][N];
int guass()
{
    int c;//当前最左边的一列
    int r;//当前最上面的一行
    for(c = 1, r = 1; c ≤ n; ++ c){
        int t = r;
        for(int i = r + 1; i ≤ n; ++ i){
            if(fabs(a[i][c]) > fabs(a[t][c]))
```

```
                t = i;
        }
        if(fabs(a[t][c]) < eps)//等于0就下一个
            continue;
        //换到第一行（最上面的一行）
        for(int i = c; i ≤ n + 1; ++ i)
            swap(a[t][i], a[r][i]);
        //把该行第一个数变成 1（该行每一列都╞═第c列的值）
        for(int i = n + 1; i ≥ c; -- i)
            a[r][i] ╞═ a[r][c];
        //中间所有列（所有项）全部消掉
        for(int i = r + 1; i ≤ n; ++ i)
            if(fabs(a[i][c]) > eps)
                for(int j = n + 1; j ≥ c; -- j)
                    a[i][j] -= a[r][j] * a[i][c];
        r ++ ;
    }
    if(r ≤ n){
        for(int i = r; i ≤ n; ++ i)
            if(fabs(a[i][n + 1]) > eps)//非零，无解
                return 2;
        return 1;//无穷多组解
    }//回代
    //行最简形矩阵
    for(int i = n; i ≥ 1; -- i)
        for(int j = i + 1; j ≤ n + 1; ++ j)
            a[i][n + 1] -= a[j][n + 1] * a[i][j];
 /* 只要保留第i行第i列的系数即可,所以从第i+1行开始一直消到第n行,
    因为只要结果,所以可以只消第i行第n+1列的数(b);就是上面消掉所有列的简化版，少了一个for循环*/
    return 0;
}
int main()
{
    cin >> n;
    for(int i = 1; i ≤ n; ++ i)
        for(int j = 1; j ≤ n + 1; ++ j)
            cin >> a[i][j];
    int t = guass();
    if(t == 0){
        for(int i = 1; i ≤ n; ++ i)
            printf("%.2f\n", a[i][n + 1]);
    }

    else if(t == 1)puts("Infinite group solutions");
    else puts("No solution");
    return 0;
}

```

0x22 高斯-约旦消元法

约旦消元法的精度更好、代码更简单，没有回带的过程。

约旦消元法大致思路如下：

- 1.选择一个**尚未被选过**的未知数作为主元，选择一个包含这个主元的方程。

- 2.将**这个方程**主元的系数化为1。
- 3.通过加减消元，消掉**其它所有方程中**的这个未知数。
- 4.重复以上步骤，直到把每一行都变成**只有一项有系数**。

我们用矩阵表示每一项系数以及结果

题目描述

给定一个线性方程组，对其求解

输入格式

第一行，一个正整数 n

第二至 $n+1$行，每行 $n+1$ 个整数，为$a_1, a_2 \cdots a_n$ 和 b，代表一组方程。

https://blog.csdn.net/weixin_45697774

```cpp
const int N = 5007;
const double eps = 1e-6;//
int n, m;
double a[N][N];

int guass() {
    for(int i = 1; i ≤ n; ++ i) {//枚举列
        int maxx = i;
        //1.找
        for(int j = i + 1; j ≤ n; ++ j)
//第i列以前的未知量除了第i一行的所有行都已经都已经消成0了，不能再动了
            if(fabs(a[j][i]) > fabs(a[maxx][i]))
                maxx = j;
        //2.换
        for(int j = 1; j ≤ n + 1; ++ j)//找到的最大一行跟第i行换
            swap(a[i][j], a[maxx][j]);
        if(fabs(a[i][i]) < eps) return -1;
        //3.减
        for(int j = 1; j ≤ n; ++ j)
//把该未知量除第i行以外全部减成0（普通的高斯是第i行以下减为0，但是需要回代）
            if(j ≠ i) {
                double tmp = a[j][i] / a[i][i];
                for(int k = i + 1; k ≤ n +1; ++ k) //只需要减第i列右边就行了，因为左边全为0
                    a[j][k] -= a[i][k] * tmp;
            }                                   //高斯 - 约旦 消元法
    }                                           //最后会得到这样的矩阵
    for(int i = 1; i ≤ n; ++ i)                 //     k1*a=e1
        a[i][n + 1] /= a[i][i];                 //     k2*b=e2
    //所以最后要除以该项的系数      ⟵          //     k3*c=e3
    return 1;                                   //     k4*d=e4
}

int main()
{
    scanf("%d", &n);
    for(int i = 1; i ≤ n; ++ i)
        for(int j = 1; j ≤ n + 1; ++ j)
            scanf("%lf", &a[i][j]);
    int ans = guass();
    if(ans == -1) puts("No Solution");
```

```
    else for(int i = 1; i ≤ n; ++ i)
            printf("%.2f\n", a[i][n + 1]);
    return 0;
}
```

方程组无解：

```
0*x1+0*x2+…0*xi+…0*xn=a(a≠0)
```

方程组无穷解

```
0*x1+0*x2+…0*xi+…0*xn=a(a==0)
```

所以对所有行在处理完后，如果
Matrix[i][i] == 0 && Matrix[i][n + 1]! = 0 就是无解
Matrix[i][i] == 0 && Matrix[i][n + 1] == 0就是无穷解。
所以我们就可以在我们上面的程序的最后加一段用于特判的代码

```
for(int i = 1;i ≤ n; ++ i) {
        if(fabs(a[i][i]) < eps)
            if(fabs(a[i][n + 1] > eps)) {
                flag = 2; //无解
                break;
            }
            else flag = 1; //无穷多组解
        else f[i] = a[i][n + 1] / a[i][i];
    if(flag == 1)
        return cout << "1" << endl, 0;
    if(flag == 2)
        return cout << "2" <<endl, 0;
}
```

需要注意的是，我们使用约旦消元法的时候，交换如果出题人不小心（）出了一组数据，所以如果出题人不小心（）出了一组SB数据，约旦消元法是会被卡掉的，虽然我们可以使用随机算法来有概率地解决这个问题，但是还是很不爽，所以尽量少使用这个算法...一般尽量使用上面的算法

0x23. 高斯消元解异或线性方程组

> 输入一个包含n个方程n个未知数的异或线性方程组。
>
> 方程组中的系数和常数为0或1，每个未知数的取值也为0或1。
>
> 求解这个方程组。
>
> 异或线性方程组示例如下：

```
M[1][1]x[1] ^ M[1][2]x[2] ^ … ^ M[1][n]x[n] = B[1]
M[2][1]x[1] ^ M[2][2]x[2] ^ … ^ M[2][n]x[n] = B[2]
…
M[n][1]x[1] ^ M[n][2]x[2] ^ … ^ M[n][n]x[n] = B[n]
```

> 其中 `^` 表示异或 (XOR)，$M[i][j]$表示第 i 个式子中 $x[j]$ 的系数， $B[i]$ 是第 i 个方程右端的常数，取值均为 0 或 1 。

异或 ⇔ 不进位的二进制加法
也就是说异或运算只会影响这一位，不会影响其他位的值

> 对于 $k = 1...n$，找到一个 $a[i][k]$ 不为 0 的行 i，把它与第 k 行交换，用第 k 行去异或下面所有 $a[i][j]$ 不为 0 的行 i，消去它们的第 k 个系数，这样就将原矩阵化成了上三角矩阵

> 由于最后一行只有一个未知数，这个未知数就已经求出来了，然后用它跟上面所有含有这个未知数的方程异或，以此类推即可以自下而上求出所有未知数。
>
> 要注意的是，对于开关问题，$a[i][j]=1$ 表示第 i 个开关受第 j 个开关影响

使用 `bitset` 优化，时间复杂度为$O(\frac{n^3}{32})$。

```cpp
const int N = 107, M = 500007, INF = 0x3f3f3f3f;
int n, m;
bitset<120> a[N];
int guass(int n, int m)//n row n 行 m col m 列
{
    int row = 0, col = 0, maxx;
    for(; col < m; ++ col) {
        for(maxx = row; maxx < n; ++ maxx)
            if(a[maxx][col])
                break;
        if(maxx == n) continue;

        if(a[maxx][col] == 0) continue;

        swap(a[maxx], a[row]);

        for(int i = row + 1; i < n; ++ i)
            if(a[i][col]) //如果这一列有数的话那一行全部消掉
                a[i] = a[i] ^ a[row];
                //for(int j = n; j ≥ col; -- j)//col前面都是0，无所谓
                    //a[i][j] = a[i][j] ^ a[row][j];
        row ++ ;
    }

    if(row < n) {
        for(int i = row; i < n; ++ i)
            if(a[i][m])//矛盾，出现非零的常数项等于0，说明无解
                return 2;
        return 1;//有无穷多组解
    }
    //行最简形矩阵，第i行第i列的表示的未知量x_i的一个解
    for(int i = n - 1; i ≥ 0; -- i) {//第i行，第i行第i列
        for(int j = i + 1; j < m; ++ j)//第j列，右边所有列都要消
                a[i][m] = a[i][m] ^ (a[j][m] * a[i][j]);
    }
    return 0;
}

int x;

itn main()
{
    scanf("%d", &n);
    m = n;
    for(int i = 0; i < n; ++ i)
        for(int j = 0; j < m + 1; ++ j) //+1是因为多了一个常数矩阵（也即是答案）
            scanf("%d", &x), a[i][j] = x;
    int res = guass(n, m);
    if(res == 0) {
        for(int i = 0; i < n; ++ i)
```

```
            x = a[i][m], printf("%d\n", x);

    }
    else if(res == 2) puts("No solution");
    else puts("Multiple sets of solutions");
    return 0;
}

```

0x24. 高斯消元解模线性方程组

在解模线性方程组组时，当有唯一解时，需要对每个方程的唯一解不断的循环取模判断，直到找出能整除的为止

```
int a[N][N];//增广矩阵
int x[N];//解集
bool freeX[N];//标记是否为自由变元
int GCD(int a, int b) {return !b ? a : GCD(b, a % b);}
int LCM(int a, int b) {return a / GCD(a, b) * b;}
int Gauss(int equ, int var) { //返回自由变元个数
    /*初始化*/
    for (int i = 0; i ≤ var; i++) {
        x[i] = 0;
        freeX[i] = true;
    }

    /*转换为阶梯阵*/
    int col = 0; //当前处理的列
    int row;//当前处理的行

    for (row = 0; row < equ && col < var; row++, col++) { //枚举当前处理的行
        int maxRow = row; //当前列绝对值最大的行

        for (int i = row + 1; i < equ; i++) //寻找当前列绝对值最大的行
            if (abs(a[i][col]) > abs(a[maxRow][col]))
                maxRow = i;

        if (maxRow ≠ row) //与第row行交换
            for (int j = row; j < var + 1; j++)
                swap(a[row][j], a[maxRow][j]);

        if (a[row][col] == 0) { //col列第row行以下全是0，处理当前行的下一列
            row--;
            continue;
        }

        for (int i = row + 1; i < equ; i++) //枚举要删去的行
            if (a[i][col] ≠ 0) {
                int lcm = LCM(abs(a[i][col]), abs(a[row][col]));
                int ta = lcm / abs(a[i][col]);
                int tb = lcm / abs(a[row][col]);

                if (a[i][col]*a[row][col] < 0) //异号情况相加
                    tb = -tb;

                for (int j = col; j < var + 1; j++)
```

```
                    a[i][j] = ((a[i][j] * ta - a[row][j] * tb) % MOD + MOD) % MOD;
            }
    }

    /*求解*/
    //无解：化简的增广阵中存在(0,0,...,a)这样的行，且a≠0
    for (int i = row; i < equ; i++)
        if (a[i][col] ≠ 0)
            return -1;

    //无穷解：在var*(var+1)的增广阵中出现(0,0,...,0)这样的行
    int temp = var - row; //自由变元有var-row个

    if (row < var) //返回自由变元数
        return temp;
    //唯一解：在var*(var+1)的增广阵中形成严格的上三角阵
    for (int i = var - 1; i ≥ 0; i--) { //计算解集
        int temp = a[i][var];
        for (int j = i + 1; j < var; j++) {
            if (a[i][j] ≠ 0)
                temp -= a[i][j] * x[j];

            temp = (temp % MOD + MOD) % MOD; //取模
        }
        while (temp % a[i][i] ≠ 0)
        //外层每次循环都是求a[i][i]，它是每个方程中唯一一个未知的变量
            temp += MOD; //a[i][i]必须为整数，加上周期MOD

        x[i] = (temp / a[i][i]) % MOD; //取模
    }
    return 0;
}
```

0x25. 矩阵求逆

【模板】矩阵求逆

求一个 $N \times N$ 的矩阵的逆矩阵。答案对 $10^9 + 7$ 取模。

学过线代的都知道

$[A, B] \Leftrightarrow [E, A^{-1}B] \leftrightarrow [A, E] \Leftrightarrow [E, A^{-1}]$

所以我们想要求矩阵的逆，只需要将矩阵和一个单位矩阵合并，然后使用高斯消元即可，最后得到的右边的矩阵就是矩阵的逆

代码：

0x26. 行列式计算

$N \times M$方阵行列式（Determinant）可以理解为所有列向量所夹的几何体的有向体积

例如：

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = 1$$

$$\begin{vmatrix} 1 & 2 \\ 2 & 1 \end{vmatrix} = -3$$

行列式有公式

$$D=|A|=\sum(-1)^v a_{1,l_1}a_{2,l_2}\dots a_{n,l_n}$$

其中v 为 $l_1,l_2\dots l_n$中逆序对的个数。

通过体积概念理解行列式不变性是一个非常简单的办法：

- 矩阵转置，行列式不变；
- 矩阵行（列）交换，行列式取反；
- 矩阵行（列）相加或相减，行列式不变；
- 矩阵行（列）所有元素同时乘以数 ，行列式等比例变大。

> 由此，对矩阵应用高斯消元之后，我们可以得到一个对角线矩阵，此矩阵的行列式由对角线元素之积所决定。其符号可由交换行的数量来确定（如果为奇数，则行列式的符号应颠倒）。因此，我们可以在$O(n^3)$的复杂度下使用高斯算法计算矩阵。
> 注意，如果在某个时候，我们在当前列中找不到非零单元，则算法应停止并返回 0。

```cpp
const double EPS = 1E-9;
int n;
vector<vector<double>> a(n, vector<double>(n));
double det = 1;
for (int i = 0; i < n; ++i) {
    int k = i;
    for (int j = i + 1; j < n; ++j)
        if (abs(a[j][i]) > abs(a[k][i]))
            k = j;
    if (abs(a[k][i]) < EPS) {
        det = 0;
        break;
    }
    swap(a[i], a[k]);
    if (i != k)
        det = -det;
    det *= a[i][i];
    for (int j = i + 1; j < n; ++j)
        a[j][i] /= a[i][i];
    for (int j = 0; j < n; ++j)
        if (j != i && abs(a[j][i]) > EPS)
            for (int k = i + 1; k < n; ++k)
                a[j][k] -= a[i][k] * a[j][i];
}
cout << det;
```

0x27. 生成树计数

一个无向图的生成树个数为邻接矩阵度数矩阵去一行一列的行列式。

详见： 矩阵树定理

例如，一个正方形图的生成树个数

$$\begin{pmatrix}0&1&0&1\\1&0&1&0\\0&1&0&1\\1&0&1&0\end{pmatrix}-\begin{pmatrix}2&0&0&0\\0&2&0&0\\0&0&2&0\\0&0&0&2\end{pmatrix}=\begin{pmatrix}-2&1&0&1\\1&-2&1&0\\0&1&-2&1\\1&0&1&-2\end{pmatrix}$$

$$\begin{vmatrix}-2&1&0\\1&-2&1\\0&1&-2\end{vmatrix}=4$$

可以用高斯消元解决，时间复杂度为 $O(n^3)$。

```cpp
#define MOD 100000007
#define eps 1e-7
struct matrix {
    static const int maxn = 20;
    int n, m;
    double mat[maxn][maxn];
    matrix() {
        memset(mat, 0, sizeof(mat));
    }
    void print() {
        cout << "MATRIX " << n << " " << m << endl;

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++)
                cout << mat[i][j] << "\t";
            cout << endl;
        }
    }
    void random(int n) {
        this->n = n;
        this->m = n;

        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                mat[i][j] = rand() % 100;
    }

    void initSquare() {
        this->n = 4;
        this->m = 4;
        memset(mat, 0, sizeof(mat));
        mat[0][1] = mat[0][3] = 1;
        mat[1][0] = mat[1][2] = 1;
        mat[2][1] = mat[2][3] = 1;
        mat[3][0] = mat[3][2] = 1;
        mat[0][0] = mat[1][1] = mat[2][2] = mat[3][3] = -2;
        this->n--;  // 去一行
        this->m--;  // 去一列
    }
    double gauss() {
        double ans = 1;
        for (int i = 0; i < n; i++) {
            int sid = -1;
            for (int j = i; j < n; j++)
                if (abs(mat[j][i]) > eps) {
                    sid = j;
                    break;
                }
            if (sid == -1)
                continue;
            if (sid != i) {
                for (int j = 0; j < n; j++) {
                    swap(mat[sid][j], mat[i][j]);
                    ans = -ans;
```

```
                }
            }
            for (int j = i + 1; j < n; j++) {
                double ratio = mat[j][i] / mat[i][i];
                for (int k = 0; k < n; k++)
                    mat[j][k] -= mat[i][k] * ratio;
            }
        }
        for (int i = 0; i < n; i++)
            ans *= mat[i][i];
        return abs(ans);
    }
};
int main() {
    srand(1);
    matrix T;
    // T.random(2);
    T.initSquare();
    T.print();
    double ans = T.gauss();
    T.print();
    cout << ans << endl;
}
```

0x30. 线性空间

0x31. 线性空间

线性空间是一个关于向量加法和向量数乘的两个运算封闭的向量集合

若给定若干个向量$a_1, a_2, \ldots, a_k$，若向量b 能由$a_1, a_2, \ldots, a_k$经过向量加法和数乘运算得出，则称向量 b 能被向量 $a_1, a_2, \ldots, a_k$ **表出**。显然，$a_1, a_2, \ldots, a_k$ 能表出的所有向量构成一个线性空间，$a_1, a_2, \ldots, a_k$被称为这个线性空间的**生成子集**，这个线性空间被称为 $a_1, a_2, \ldots, a_k$ 的子空间。

任意选出线性空间的若干个向量，如果其中存在一个向量能被其他是向量线性表出，则称这些向量**线性相关**，否则称这些向量线性无关。

线性无关的生成子集被称为线性空间的**基底**，简称为**基**。基的另一种定义是线性空间的极大线性无关子集，一个线性空间的所有的基包含的向量个数相等，这个数被称作线性空间的**维数**。

0x32. 线性相关

n维线性空间：将其看做一个集合，其中的每一个元素均使用n个相互独立的自由变量表示，如坐标系可看做二维线性空间

线性组合：设$\alpha_1, \alpha_2, \ldots, \alpha_e (e \geq 1)$是域 P 上线性空间 V 中的有限个向量.若V中向量 α 可以表示为：$\alpha = k_1\alpha_1 + k_2\alpha_2 + \ldots + k_e\alpha_e (k_e \in P, e = 1, 2, \ldots, s)$,则称α是向量组$\alpha_1, \alpha_2, \ldots, \alpha_e$ 的一个线性组合,亦称 α 可由向量组 $\alpha_1, \alpha_2, \ldots, \alpha_e$ 线性表示或线性表出

线性相关性

对于一个 n 维向量元素集合$S(a_1, a_2 \ldots, a_m)$，若存在等式$a_1k_1 \times a_2k_2 \ldots \times a_mk_m = 0$（$k_1 \ldots k_m$不全为 0），则称 A 线性相关，反之线性无关

等价向量组

对于向量组A：$a_1,a_2,\dots,a_m$、B：$b_1,b_2,\dots,b_n$，如果 A 能被 B 线性表示且 B 能被 A 线性表示，则向量组 A、B 等价

等价向量组定理：若 A 可由 B 线性表示且 $m>n$，则 A 线性相关

推论1：若 A 可由 B 线性表示且$m\leq n$，则 A 线性无关

推论2：两个线性无关的等价向量组，包含**个数相同的向量**

0x33. 极大线性无关组

对于向量组A：$a_1\dots a_n$ ，若存在一组 $a_i\dots a_r(r-i<n)$ 线性无关且 $a_i\dots a_{r+1}$ 线性相关，则称 $a_i\dots a_r$ 为 A 的一组极大线性无关组，通俗的说，极大线性无关组 $a_i\dots a_r$ 能表示向量组 A 中的其它任意元素且本身线性无关

- 若A线性无关，则其极大线性无关组就是其本身

极大无关组性质：

- 向量组A的任意极大线性无关组与A等价
- 向量组A的任意两个极大线性无关组等价

推论：一个向量组A的任意两个极大线性无关组等价等大

0x34. 向量组的秩

A 的极大线性无关组的大小称为向量组 A 的秩，记作 $R(A)$

性质：若向量组 A 可由向量组 B 线性表示，则$R(A)\leq R(B)$

0x35. 线性基

$2^{20}\approx 10^6$
$2^{24}\approx 10^7$
$2^{27}\approx 10^8$
$2^{30}\approx 10^9$
$2^{34}\approx 10^{10}$
$2^{60}\approx 10^{18}$
上述的约等于均是大于（往下看就知道有什么用了）

0x35.1 前置知识

异或和

设 S 为一无符号整数集（下文中除非特殊说明，集合均指「无符号整数集」），定义其异或和 $\text{xor-sum}(S)=S_1\text{ xor }S_2\text{ xor }\dots\text{ xor }S_S$。

张成

设 $T\subseteq S$，所有这样的子集 T 的异或和组成的集合称为集合 S 的张成，记作 $\text{span}(S)$ 。即，在 S 中选出任意多个数，其**异或和的所有可能的结果**组成的**集合**。

线性相关

对于一个集合 S ，如果存在一个元素 S_j ，使得，S 在去除这个元素后得到的集合 S' 的张成 $\text{span}(S)$ 中包含 S_j ，即，$S_j\in\text{span}(S')$，则称集合 S 线性相关。

更形象地，可以表示为，存在一个元素 S_j，可以用其它若干个元素异或起来得到。

相对的，如果不存在这样的元素 S_j，则称集合 S **线性无关**。

一个显然的结论是，对于这个线性相关的集合 S ，去除这个元素后，集合的张成不变。

线性基

线性基是向量空间的一组基。

我们称集合 B 是集合 S 的线性基，当且仅当：

- $S \subseteq \text{span}(B)$，即 集合$S$ 是 集合B 的张成的子集；
- 集合B是线性无关的。

集合 B 中元素的个数，称为线性基的**长度**，也就是该线性空间的维数。

线性基有以下基本性质：

- B是极小的满足线性基性质的集合，它的任何真子集都不可能是线性基；
- S 中的任意元素都可以唯一表示为 B 中若干个元素异或起来的结果。
- 线性基的元素能相互异或得到原集合的元素的所有相互异或得到的值，并且线性基是满足该性质的最小的集合。
- 线性基没有异或和为 0 的子集。
- 线性基中每个元素的异或方案唯一，也就是说，线性基中不同的异或组合异或出的数都是不一样的。（因为前面也说了，基是极大线性无关向量组）
- 线性基中每个元素的二进制**最高位互不相同**。

那么如何求得线性基呢？

0x35.2 求线性基

两个题目

- 查询最大异或和

> 题目大意： 给定n个整数（数字可能重复），求在这些数中选取任意个，使得他们的异或和最大。
> $n \leq 10^5, 0 \leq s_i \leq 2^{63}-1$

- 求第 k 小异或和

> 给定你由N个整数构成的整数序列，你可以从中选取一些（**至少一个**）进行异或（XOR）运算，从而得到很多不同的结果。
>
> 请问，所有能得到的不同的结果中第k小的结果是多少。$n \leq 10^4, k \leq 10^{18}$。

考虑将 n 个数在二进制下看成 n 个向量，每个向量中的每个元素仅有0、1两种取值。

如果向量 X 能被 $a_1, a_2..a_n$ 这些向量通过异或运算得到，则称 X 能被 $a_1, a_2..a_n$ 表出。如果某个向量集合中存在若干个向量能被其它向量表出，则这些向量线性相关，否则线性无关

定义一个向量集合的线性基为其极大线性无关子集（可能会有多个线性基），显然线性基可以表出这个向量集合

线性异或空间是对异或运算封闭的向量集合。

异或高斯消元仅有向量间的异或操作，所以消元不会改变这些向量能表出的线性空间。因此，高斯消元可以求得一组线性基。

我们用高斯消元求出的$a_1, a_2, \ldots, a_n$构成的异或空间的基，我们设这个基有整数$b_1, b_2, \ldots, b_t$构成，其中 $b_1 > b_2 > \cdots > b_t$，从基底中选取若干个整数，显然有2^t种选法，因为选或者不选，每个数两种取法，因此这个 t 维线性异或空间一共有 2^t 个整数，与这 2^t 种取法**一一对应**。（这里是关键）

在上面的高斯消元的过程中，一共有 t 个主元，分别为 $b_1, b_2, \ldots, b_t$ 的最高位，假设它们的最高位分别是第 $c_1, c_2, \ldots, c_t$，那么显然 $c_1 > c_2 > \cdots > c_t$。

由高斯消元得到的**线性基**一定是**行最简形型矩阵**，即在阶梯形矩阵中，若非零行的第一个非零元素全是1，且非零行的第一个元素1所在列的其余元素全为零，就称该矩阵为行最简形矩阵。

例如整数$5, 12, 2, 7, 9$，写成的 01 矩阵，以及高斯消元后得到的简化阶梯型矩阵如下：

```
logs 列 n 行（s是数组最大值，n是数组个数）
0 1 0 1    1 0 0 1
1 1 0 0    0 1 0 1
0 0 1 0 ⇒ 0 0 1 0
0 1 1 1    0 0 0 0
1 0 0 1    0 0 0 0
```

异或空间的基是$9,5,2$，最高位1分别在第3位，第2位，第1位，自由元是第0位，可以发现，只有9的第3位是1，5和2的第3位都是0，即满足主元所在位为 1 的整数是唯一的。

因为基底 $b_1,b_2,\ldots,b_t$ 能与 $a_1,a_2,\ldots,a_n$线性表出相同的线性异或空间，所以异或空间种第 k 小的整数，就是从基底 $b_1,b_2,\ldots,b_t$ 中选出若干个数做异或运算，能线性表出的第 k 小的整数，也就是整个线性空间里第 k 小的整数。而第 c_i 位是 1 的只有 b_1，所以选择 b_1 一定比不选 b_1 所线性表示出来的数更大，同理可得，整个线性异或空间中**最大的数**就是 $b_1>b_2>\cdots>b_t$全部异或起来得到的数，也就是从 n 个数中选若干个数字能构成的最大的异或和。

所以我们的答案一定是主对角线上的元素一全部异或在一起是最优的。

根据上面的性质，n个整数在二进制下组成的向量组的线性空间的基的维数只有63，因为二进制下$s_i\le 2^{63}-1$，最多有62位，也就是说最多有63 个二进制位数，所以**线性基包含的向量最多只有63个**，所以整体的时间复杂度为$O(nlogs)$。

第一题代码：

```cpp
const int N = 200007, mod = 1e9 + 7, INF = 2.1e9;
const double eps = 1e-8;
int n, m;
ll a[N];
int main()
{
    scanf("%d", &n);
    for(int i = 0; i < n; ++ i)
        scanf("%lld", &a[i]);
    int k = 0;
    //每个数就是一个二进制的向量
    //先枚举列，也就是二进制，最多62位
    //做一次高斯消元
    for(int i = 62; i ≥ 0; -- i) {
        for(int j = k; j < n; ++ j)
            if(a[j] >> i & 1){//第i位是1
                swap(a[j], a[k]);//相当于整个向量交换
                break;
            }
        if(!(a[k] >> i & 1)) //如果这一位全是0，就跳过
            continue;
        for(int j = 0; j < n; ++ j)
            if(j ≠ k && (a[j] >> i & 1))
            a[j] ^= a[k];//垄断，除了找到的k以外全部变成0
        k ++ ;
        if(k == n) break;
    }

    ll res = 0;
    for(int i = 0; i < k; ++ i)
    //最后上三角形的主对角线的k个数异或起来就是答案，因为每一个数都要选才会最优
        res ^= a[i];
    printf("%lld\n", res);
    return 0;
```

```
35 }
```

继续考虑第二题

题目中不允许出现$a_i \text{ xor } a_i$，也就是不一定能线性表出 0，而普通的线性空间是允许的，导致普通线性空间一定存在整数 0，而从$a_1, a_2, a_3 \ldots, a_n$中选出不同的数异或起来，可能得不到 0。

我们这里设base是线性基的维数
当 base < n时，则向量组里这 n 个向量线性相关，一定可以构成0。
因为若线性相关，一定存在

$$x_i = a_1x_1 + a_2x_2 + \ldots a_nx_n$$

则

$$0 = a_1x_1 + a_2x_2 + \ldots a_nx_n - x_i$$

即可以线性表出整数 0。

若 base = n，则则向量组里这 n 个向量线性无关，一定不能线性表出整数 0。

因为若可以表出 0，则一定存在

$$0 = a_1x_1 + a_2x_2 + \ldots a_nx_n$$

若a_1非零，则

$$-a_1x_1 = a_2x_2 + \ldots a_nx_n$$

$$x_1 = -\frac{a_2}{a_1}x_2 - \cdots - \frac{a_n}{a_1}x_n$$

存在 x_1 可以由向量组的其他向量线性表出，说明向量组一定线性相关，与前提条件矛盾，故当向量组线性无关，即 base = n 时，不能线性表出 0。

所以综上所诉，如果能得到 0 的话，就把 $k-1$ 进行二进制分解，减掉 1 是为了把最小的数定义为第 0 小（因为我们二进制有第 0 位），题目中是第 1 小，如果$k-1$的第$j(0 \le j \le t)$位等于1，就选取 b_j，也就是从左往右数第 j 个（二进制是从右往左数，所以我们重新从小到大，从右往左循环vector存一下 b 数组999），最后把所有选出的 b 异或起来，就是答案。特别地，若 $k > 2^t$，则无解。

这样选的原因是因为上面说了，如果我们线性基有 t 个向量，那么就会有 2^t 个取法，也就是说我们要找的第 k 小的数和上面 2^t 个数一一对应，因为线性基的$b_1, b_2, \ldots, b_t$ 已经从大到小排过序了，所以同样的顺序选取即可，根据二进制的性质，正确性显然。

如果不能线性表出0的话就把 k 进行二进制分解，而不是 $k-1$，相当于统计的时候把 0 跳过去，因为此时应该没有 0，但是线性基中一定有 0，所以我们假装看不见它。

~~啊，感觉说的有点乱...~~

```
1
2     ll query_kth(ll k, int n) {
3         ll ans = 0;
4         if(zero) k -- ;
5         if(k ≥ (1ll << (int)mmap.size()))
6             return -1;
7         for(int i = 0; i < (int)mmap.size(); ++ i)
8             if((k >> i) & 1)
9                 ans ^= mmap[i];
10        return ans;
11    }
12 };
```

0x35.3 线段树套线性基

> 给你 n 和 n 个整数的数组 a，以及k和q，有 q 次询问，每次询问输入两个整数l,r，表示区间$[l,r]$，要你每次在这个区间里选择若干个数，使得这些数的异或和sum，再按位或k的值最大，即求最大的 `res = sum | k`。详细题解（2017年ICPC西安邀请赛A）

将 k **取反**，然后把所有数在加入线性基之前，全部 **按位异或运算一遍**，这样把没用的1全部删掉，再加入线性基。最终的答案就是 `k | sum` （`sum` 是修改后的区间的数插入线性基的最大异或和）。我们使用线段树维护区间操作即可。实际上就是把线段树权值换成当前区间的线性基，简单维护一下即可。

```cpp
const int N = 50007, mod = 1e9 + 7, INF = 2.1e9;
const double eps = 1e-8;
const int Base = 27 * 2;//线性基数组大小，开大一点，嘻嘻嘻
struct LinearBase {
    int dimension = 27; // dimension 维数，就是线性基的维数 = logs, (s为元素最大值)
    //dimension一定要是logs,错一点就会WA呜呜呜
    //线性基数组
    ll a[Base + 7];
    LinearBase() {fill(a, a + Base + 7, 0);}
    LinearBase(ll *x, int n) {build(x, n);}
    void insert(ll t)
    {
        // 逆序枚举二进制位
        for(int i = dimension; i ≥ 0; -- i) {
            if(t == 0) return ;
            // 如果 t 的第 i 位为 0，则跳过
            if(!(t >> i & 1)) continue;
            // 如果 a[i] ≠ 0，则用 a[i] 消去 t 的第 i 位上的 1
            if(a[i]) t ^= a[i];
            else {
                // 找到可以插入 a[i] 的位置
                for(int k = 0; k < i; ++ k)
                    if(t >> k & 1) t ^= a[k];
                // 用 t 消去 a[i + 1 ... L] 的第 i 位上的 1
                for(int k = i + 1; k ≤ dimension; ++ k)
                    if(a[k] >> i & 1) a[k] ^= t;
                // 插入到 a[j] 的位置上
                a[i] = t;
                break;
            }// 此时 t 的第 i 位为 0，继续寻找其最高位上的 1
        }// 如果没有插入到任何一个位置上，则表明 t 可以由 a 中若干个元素的异或和表示出，即 t 在 span(a)中
    }
    // 数组 x 表示集合 S，下标范围 [1 ... n]
    void build(ll *x, int n)
    {
        fill(a, a + Base + 7, 0);
        for(int i = 1; i ≤ n; ++ i)
            insert(x[i]);
    }

    ll query_max()
    {
        ll res = 0;
        for(int i = 0; i ≤ dimension; ++ i)
            res ^= a[i];
```

```
        return res;
    }

    void mergefrom(const LinearBase &other) {
        for(int i = 0; i ≤ dimension; ++ i)
            insert(other.a[i]);
    }

    static LinearBase merge(const LinearBase &a, const LinearBase &b) {
        LinearBase res = a;
        for(int i = 0; i ≤ 27; ++ i)
            res.insert(b.a[i]);
        return res;
    }
};
int n, m, t, q, k;
ll a[N];

struct Tree {
    int l, r;
    LinearBase elem;//element
}tr[N];

void build(int p, int l, int r)
{
    tr[p] = {l, r};
    if(l == r) {
        tr[p].elem.insert(a[r]);
        return ;
    }
    int mid = (l + r) >> 1;
    build(p << 1, l, mid);
    build(p << 1 | 1, mid + 1, r);
    tr[p].elem = tr[p].elem.merge(tr[p << 1].elem, tr[p << 1 | 1].elem);
}

LinearBase query(int p, int l, int r)
{
    if(l ≤ tr[p].l && tr[p].r ≤ r)
        return tr[p].elem;
        int mid = (tr[p].l + tr[p].r) >> 1;
        LinearBase res;
        if(l ≤ mid) res = res.merge(res, query(p << 1, l, r));
        if(r > mid) res = res.merge(res, query(p << 1 | 1, l, r));
        return res;
}

int main()
{
    scanf("%d", &t);
    while(t -- ) {
        scanf("%d%d%d", &n, &q, &k);
        for(int i = 1; i ≤ n; ++ i) {
            scanf("%lld", &a[i]);
```

```
            /*k的存在会对求线性基最大值时的主元产生影响，所以预处理一下，a[i]只保留k为0的位，这样贡献最大
    */
            for(int j = 0; j < 27; ++ j) {
                if((k >> j & 1) && (a[i] >> j & 1))
                    a[i] ^= (1 << j);
            }
        }
        build(1, 1, n);
        while(q -- ) {
            int l, r;
            scanf("%d%d", &l, &r);
            LinearBase res = query(1, l, r);
            printf("%lld\n", res.query_max() | k);
        }
    }
    return 0;
}
```

0x35.6 线性基全家桶（code）

支持操作：

- 合并

> 两个集合的线性基可以在 $O(log^2t)$的时间内进行合并，合并后得到的线性基为两个集合的并的线性基。
> 合并只需要将一个线性基中的所有元素插入到另一个线性基中即可。

- 求最大异或和
- 求最小异或和
- 求第k小值
- 检查某数是否能被线性表出中
  （即是否在线性空间中）
- 查询某数在线性空间中的排名
- 线性基中动态插入一个数
- 合并两个线性基
  以上操作均为 $O(logs)$

```
#include <cstdio>
#include <iostream>
#include <algorithm>
#include <cstring>
#include <cmath>
#include <map>
#include <queue>
using namespace std;
typedef long long ll;
typedef int itn;
typedef pair<int, int>PII;
const int N = 200007, mod = 1e9 + 7, INF = 2.1e9;
const double eps = 1e-8;
const int Base = 60 * 2;//线性基数组大小，开大一点，嘻嘻嘻

int base_cnt;
bool zero;

struct LinearBase
{
```

```
    int dimension = 60; // dimension 维数，就是线性基的维数 = logs, (s为元素最大值)
    //dimension一定要是logs,错一点就会WA呜呜呜
    //线性基数组
    ll a[Base + 7];//注意这里要加一点，因为下面的循环也是这个数
    vector<ll> mmap;
    LinearBase() {/*构造方法*/
        //for(int i = 0; i ≤ Base; ++ i)
        //    a[i] = 0;
        fill(a, a + Base + 7, 0);
    }

    LinearBase(ll *x, int n) {build(x, n);}

    void insert(ll t)/*插入一个数*/
    {
        // 逆序枚举二进制位
        for(int i = dimension; i ≥ 0; -- i) {
            //if(t == 0) return ;
            // 如果 t 的第 i 位为 0，则跳过
            if(!(t >> i & 1)) continue;
            // 如果 a[i] ≠ 0，则用 a[i] 消去 t 的第 i 位上的 1
            if(a[i]) t ^= a[i];
            else {
                // 插入到 a[i] 的位置上

                // 找到可以插入 a[i] 的位置
                // 用 a[0 ... i - 1] 消去 t 的第 [0, i) 位上的 1
                // 如果某一个 a[k] = 0 也无须担心，因为这时候第 k 位
                //不存在于线性基中，不需要保证 t 的第 k 位为 0
                for(int k = 0; k < i; ++ k)
                    if(a[k] && (t >> k & 1)) t ^= a[k];
                // 用 t 消去 a[i + 1 ... L] 的第 i 位上的 1
                for(int k = i + 1; k ≤ dimension; ++ k)
                    if(a[k] >> i & 1) a[k] ^= t;
                a[i] = t;
                base_cnt ++ ;
                break;
            }
            // 此时 t 的第 i 位为 0，继续寻找其最高位上的 1
        }
        // 如果没有插入到任何一个位置上，则表明 t 可以由 a 中若干个元素的异或和表示出，即 t 在 span(a)
中
    }
    // 数组 x 表示集合 S，下标范围 [1 ... n]
    void build(ll *x, int n)/*求整个数组（向量组）的线性基*/
    {
        base_cnt = 0;
        zero = 0;
        fill(a, a + Base + 7, 0);
        for(int i = 1; i ≤ n; ++ i)
            insert(x[i]);
        zero = base_cnt ≠ n;/*是否有零行，表示是否能够线性表出0*/
        mmap.clear();
        for(int i = 0; i ≤ dimension; ++ i)
            if(a[i])
```

```
75                  mmap.push_back(a[i]);//逆序从第0位存到第t位，这样下标可以直接对应，比较方便
76      }
77
78      ll query_max()/*求最大异或和*/
79      {
80          ll res = 0;
81          for(int i = 0; i ≤ dimension; ++ i)
82              res ^= a[i];
83          return res;
84      }
85
86      ll query_min(int n)/*求最小异或和，至少选择一个数*/
87      {
88          zero = base_cnt ≠ n;
89          if(zero) return 0;
90          for(int i = 0; i ≤ dimension; ++ i)
91              if(mmap[i])
92                  return mmap[i];
93      }
94
95      bool check_in(ll x)/*check一个数是否在线性基中*/
96      {
97          for(int i = dimension; i ≥ 0; -- i)
98              if(x >> i & 1)
99                  x ^= a[i];
100         return x == 0;
101     }
102
103     void mergefrom(const LinearBase &other) {/*一个线性基与另一个线性基合并*/
104         for(int i = 0; i ≤ dimension; ++ i)
105             insert(other.a[i]);
106     }
107
108     static LinearBase merge(const LinearBase &a, const LinearBase &b) {/*合并两个线性基，多用于线
    段树维护线性基*/
109         LinearBase res = a;
110         for(int i = 0; i ≤ 60; ++ i)
111             res.insert(b.a[i]);
112         return res;
113     }
114
115     ll query_kth(ll k, int n) {/*查询线性空间中第k小的值*//*细节颇多，注意zero的特判*/
116     /*这里别忘了在主函数里初始化base_cnt = 00以及zero = 0，已在build函数中初始化，所以数组尽量直接用
    build*/
117         ll ans = 0;
118         if(zero) k -- ;
119         if(k ≥ (1ll << (int)mmap.size()))
120             return -1;
121         for(int i = 0; i < (int)mmap.size(); ++ i)
122             if((k >> i) & 1)
123                 ans ^= mmap[i];
124         return ans;
125     }
126
127     ll query_rank(ll x) {
```

```
        ll ans = 0;
        for(int i = 0; i ≤ (int)mmap.size(); ++ i)
            if(x ≥ mmap[i])
                ans += (1ll << i), x ^= mmap[i];
        return ans + zero;
    }
};//上述操作均通过简单测试
int n;
ll x;
ll cnt, t, q, k;
ll a[N];

int main()
{
    scanf("%d", &t);
    while(t -- ) {
        scanf("%d", &n);
        for(ll i = 1; i ≤ n; ++ i)
            scanf("%lld", &a[i]);

        LinearBase Line(a, n);

        printf("Case #%d:\n", ++ cnt);
        scanf("%lld", &q);

        while(q -- ) {
            scanf("%lld", &k);
            printf("%lld\n", Line.query_kth(k, n));
        }
    }
    return 0;
}
```

- 求两个线性基的交
- 求两个线性基的并（通过容斥原理将并转化为交即可）

两个线性空间 V_1 与 V_2，求出这两个空间的交与并。当然OI中的线性空间大部分是指异或操作下的，也就是常说的线性基。
如下所说的，都是异或的线性基，不区分加号与异或，即$\sum x_i = x_1 \oplus x_2 ... \oplus x_3$ 。
当然可以扩展到任意线性空间，大致方法不变。

明显答案也是一个线性基。
那么我们还需要如下引理：
设 $\{\alpha_{i=1-x}\}$ 是 V_1 的一组线性基，$\{\beta_{i=1-y}\}$ 是 V_2 的一组线性基。设W WW是 $\{\beta\}$ 中在 V_1 的基，如果$\alpha \bigcup \{\beta \setminus W\}$ 线性无关，那么 W 就是交的线性基。
从而可以得到：

如果β_i 能被$\{\alpha_{i=1-x}\}$ 线性表示出来，我们可以把$\bigoplus_{i=1}^{x} \alpha_i$ 或者 $\bigoplus_{j=1}^{i-1} \beta_j$
（注意要么加入 α 要么加入 β ）加入答案的线性基中。
code：

```
LinearBasis Merge(LinearBasis A,LinearBasis B) {
    LinearBasis All , C , D;
    All.clear();
    C.clear();
    D.clear();
```

```
    for (int i = 60;i ≥ 0;i--) {
        All.basis[i] = A.basis[i];
        D.basis[i] = 1ll << i;
    }
    for (int i = 60; i ≥ 0; i--) {
        if (B.basis[i]) {
            ll v = B.basis[i] , k = 0;
            bool can = true;
            for (int j = 60; j ≥ 0; j--) {
                if (v & (1ll << j)) {
                    if (All.basis[j]) {
                        v ^= All.basis[j];
                        k ^= D.basis[j];
                    } else {
                        can = false;
                        All.basis[j] = v;
                        D.basis[j] = k;
                        break;
                    }
                }
            }

            if (can) {
                ll v = 0;
                for (int j = 60; j ≥ 0; j--) {
                    if (k & (1ll << j)) {
                        v ^= A.basis[j];
                    }
                }
                C.insert(v);
            }
        }
    }
    C.build();
    return C;
}

```

0x35.7 线性基总结

线性基的题型相对比较固定，关于异或的运算基本上都是线性基了：

- 最大异或和
- 第 k大异或和/异或和是第几大
- 求所有异或值的和

线性基中的数记得开 `long long`
线性基的维数为 $logs$，s 为数组中最大的值，用于 `for` 循环的上界。千万不能开大不然会WA

一个小技巧：

任意一条 1 到 n 的路径的异或和，都可以由任意一条 1 到 n 路径的异或和与图中的一些环的异或和来组合得到。

0x40. 最后的话

整理了两三天了，近三万字的全家桶，点个赞 (关注) 再走呗

部分内容参考李超2013年的WC课件，里面列举到的题目（Problem系列）有的是BZOJ上的题（BZOJ没了呜呜呜）有的应该是李超的原创题，找不到原题，有谁知道哪题的原题的话评论区里说一声呗 QwQ

参考资料：

- 李超（北京大学）线性代数在OI中的应用与题目讲解
  （需要百度文库VIP，我充钱下载下来了，有需要找我）
- OI中的线性代数
- ACM中的线性代数基础
- 高斯消元快速入门
- 线性代数 —— 高斯消元法
- oiwiki
- 关于矩阵在ACM中的应用
- 《线性代数》概念定理大全
- 线性代数大全
- 线性基学习笔记
- 算法进阶课 y总yyds
- 线性基学习笔记